Forecasting Lake Michigan–Huron Water Levels with Machine Learning: A Comparison with Physics-Based Models

Samundra Pathak
University of Wisconsin–Green Bay

October 20, 2026

Abstract

Machine learning and physics-based models were built and evaluated to forecast the monthly mean water level of Lake Michigan–Huron one to six months ahead, using 74 years of public data (1950–2023) from the National Oceanic and Atmospheric Administration (NOAA) and the Great Lakes Observing System (GLOS). The monthly over-lake precipitation, evaporation, river runoff and lake level were merged into one table. Two machine learning models, gradient boosting and a physics-guided convolutional neural network (CNN), were compared with persistence, a seasonal climatology, a water-balance regression built on the physics of the lake, a more detailed physical lake model with a curved outflow, and a hybrid in which the trees correct the errors of the water-balance model. Since this is time series data, a rolling-origin backtest was used, in which each of the 44 test years (1980–2023) was forecast by models trained only on the years before it. The water-balance model reached a root-mean-square error (RMSE) of 3.22 cm at one month and 14.66 cm at six months. Gradient boosting was slightly worse at every horizon (3.32 cm and 15.39 cm), but the differences were within the sampling uncertainty, and the hybrid was no better than the water-balance model alone. The more detailed lake model was also slightly worse (3.26 cm and 14.78 cm), and the CNN, which corrects the weather forecast that drives the lake model, stayed almost identical to it (3.26 cm and 14.76 cm). When the water-balance model was given the precipitation, evaporation and runoff that actually happened, its error dropped to 2.39 cm and 8.49 cm. This result shows us that the limit on forecast skill is the unknown future weather, not the flexibility or the detail of the model. In the final section, we discuss the implications of these findings and suggest ways to improve the forecasts.

1 Introduction

The water level of the Great Lakes matters to a lot of people. Shoreline property owners, shipping, harbors and wetlands all depend on it, and it can move a long way in only a few years. Lake Michigan–Huron reached its lowest level on record in January 2013. Only seven and a half years later, in July 2020, it was 1.88 m higher and close to its record high. The rise in 2013 and 2014 alone was the fastest ever recorded on the lake over a two-year period [1], and water levels across all of the Great Lakes kept rising for several years after this period of record lows [2]. Being able to forecast the level a few months ahead is useful for anyone who has to plan around it.

Forecasting the lake level looks like a natural problem for machine learning, because long public records exist for the level and for the water entering and leaving the lake. But unlike a stock price, the lake level has well-understood physics behind it. The lake works like a bathtub: the level changes only because water is added to it or removed from it. In this case study, that fact is used as a test. A model built around the water balance of the lake is compared with a flexible machine learning model that is given the same information and has to find the structure by itself. Two further models sit between these extremes: a more detailed physical model of the lake, and a neural network that has to work through that physical model.

The lake that is used here is Lake Michigan–Huron. Lakes Michigan and Huron are joined at the Straits of

Mackinac and stand at the same elevation, so hydraulically they behave as one lake with a surface area of about 117,400 km^2 [3]. The goal of this case study is to build a complete forecasting pipeline and to answer one narrow question: when forecasting the monthly mean level one to six months ahead, does machine learning beat simple baselines, and does it beat a physics-based water-balance model? Two follow-up questions are also asked: does a more detailed physical model do better, and does a neural network do better when it is made to respect the physics? Building knowledge of the physical process into a neural network in this way has improved predictions of lake water temperature [4]. The comparison is meant to be honest, including the case where machine learning does not win.

2 Data

2.1 Source

All of the data used in this study is public. The monthly over-lake precipitation, over-lake evaporation and river runoff for Lake Michigan and Lake Huron were retrieved from the Great Lakes Monthly Hydrologic Data of the NOAA Great Lakes Environmental Research Laboratory (GLERL) [5], a database described by Hunter et al. [6]. The monthly lake-wide mean water level was retrieved from the Seagull ERDDAP server of GLOS [7]. Levels are given in meters above the International Great Lakes Datum of 1985 (IGLD 1985), the reference height used for all Great Lakes levels. In total, seven raw files were used (Table 1), all of them downloaded on October 12, 2026. The raw files were never modified; every change to the data was done in code.

Table 1: The seven raw files used in this study.

Variable	Source	Files	Units	Period with data
Over-lake precipitation	NOAA GLERL	Michigan, Huron	mm per month	Jan 1940 – Dec 2024
Over-lake evaporation	NOAA GLERL	Michigan, Huron	mm per month	Jan 1940 – Dec 2024
Basin runoff	NOAA GLERL	Michigan, Huron	m^3/s	1901 – Aug 2024
Lake level	GLOS	Michigan–Huron	m (IGLD 1985)	Jan 1918 – Dec 2023

The download links for both sources are given in the references. The surface areas of the two lakes were taken from the U.S. Environmental Protection Agency [3].

Two things were checked before the files were used. First, the level file is labeled "Huron". Since Michigan and Huron are one lake, it should be the level of the combined lake, but that was worth checking. Its lowest value, 175.57 m in January 2013, and its highest value, 177.50 m in October 1986, are the known record low and record high of Lake Michigan–Huron, so the series was used as the level of the combined lake. Second, Lake Michigan evaporation for 1941–1944 is almost the same from one year to the next, which real weather never is. These years look filled in with modeled or average values, so the analysis starts in January 1950. The level file runs to October 2025 but has no values after December 2023, so the analysis ends there. Figure 1 shows the level record and the average seasonal cycle of the supply.

2.2 Building the Monthly Table

The raw files come in different shapes and units. The precipitation and evaporation files have one row per year with twelve monthly columns, while the runoff and level files have one row per month. A few steps were needed to turn them into one table with one row per month:

- **Combining the Two Lakes**: Precipitation and evaporation are given as a depth of water over each lake. Since Michigan and Huron are treated as one lake, the two depths were combined as an area-weighted average,

$$P_t = w_M \, P_t^{\mathrm{Mic}} + w_H \, P_t^{\mathrm{Hur}}, \tag{1}$$

$$w_M = \frac{A_M}{A_M + A_H}, \qquad w_H = 1 - w_M, \tag{2}$$

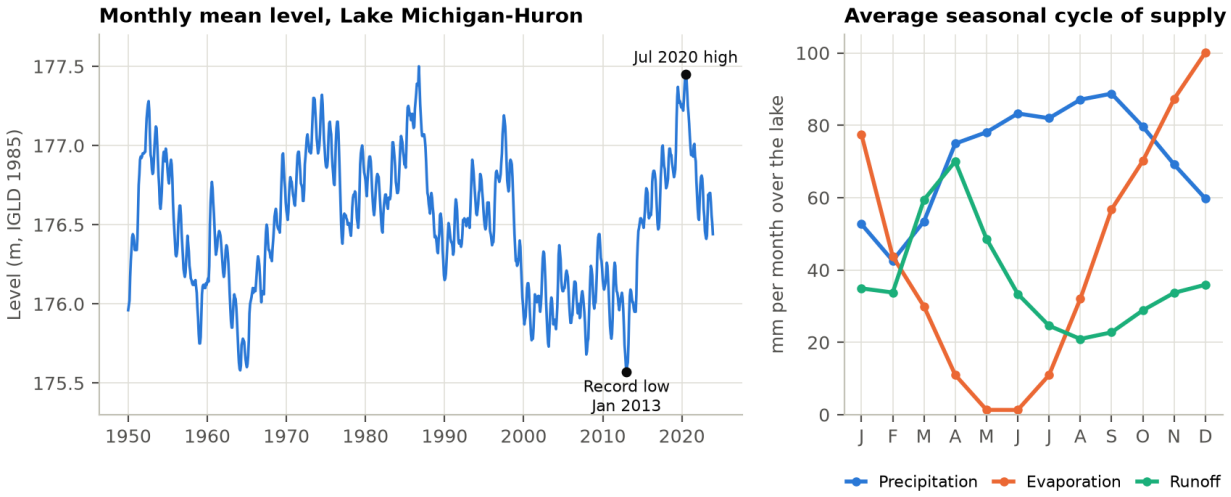


Figure 1: Left: monthly mean level of Lake Michigan–Huron, 1950–2023, with the January 2013 record low and the July 2020 high marked. Right: average seasonal cycle of precipitation, evaporation and runoff over the lake. Runoff peaks with the spring snowmelt and evaporation peaks in late autumn and early winter.

and the same was done for evaporation E_t. With $A_M = 57{,}800\,\mathrm{km}^2$ and $A_H = 59{,}600\,\mathrm{km}^2$, the weights are $w_M = 0.492$ and $w_H = 0.508$.

- **Runoff as a Depth**: Runoff is the river water draining from the land into the lake. It is given as a flow in m^3/s for each basin, not as a depth. Flows add, so the two basins were summed first. The total flow was then multiplied by the number of seconds in the month to get a volume, and that volume was spread over the lake surface:

$$R_t = \frac{\left(Q_t^{\mathrm{Mic}} + Q_t^{\mathrm{Hur}}\right) \times N_t \times 86{,}400}{A_M + A_H} \times 1000, \tag{3}$$

where N_t is the number of days in month t and the areas are in m^2. As a check on the scale, a typical combined runoff of $1{,}500\,\mathrm{m}^3/\mathrm{s}$ over a 30-day month is 33 mm over the lake. Because the lake is so large, one month of supply moves the level by only a few centimeters.

- **Net Basin Supply (NBS)**: Now that all three terms are depths in mm, they can be added. The result is the net basin supply, one of the standard hydrologic quantities for the Great Lakes [5]:

$$\mathrm{NBS}_t = P_t - E_t + R_t. \tag{4}$$

NBS is the amount the lake would rise in month t if no water flowed in or out through the rivers that connect it to the other lakes.

- **Monthly Level Change**: The change in the monthly mean level from the previous month, in mm:

$$\Delta L_t = 1000\,(L_t - L_{t-1})\,. \tag{5}$$

It can be noticed that the level record is a series of monthly means, not end-of-month values. Suppose the level rises at a steady rate s_t during month t, and let ℓ be the level at the boundary between months $t-1$ and t. The mean level in month t is then $\ell + \frac{1}{2}s_t$, and the mean level in month $t-1$ is $\ell - \frac{1}{2}s_{t-1}$. Subtracting the two gives

$$\Delta L_t = \tfrac{1}{2}\,s_{t-1} + \tfrac{1}{2}\,s_t. \tag{6}$$

This small fact is important to us because it shows that the change between two monthly means depends equally on the supply of this month and of the month before. It is the reason the water-balance model in Section 3.2 uses two months of every supply term.

Rows before 1950 and after the last month with every series present were then dropped. The final table has 888 months, from January 1950 to December 2023, with no gaps and no missing values. Table 2 summarizes the notation used in Equations 1–6.

Table 2: Notation used in building the monthly table.

Symbol	Meaning
t	Index of a month (a row in the table)
L_t	Monthly mean lake level in month t (m)
ΔL_t	Change in the monthly mean level from month $t-1$ to month t (mm)
$P_t^{\mathrm{Mic}},\ P_t^{\mathrm{Hur}}$	Precipitation over Lake Michigan and over Lake Huron (mm)
$P_t,\ E_t,\ R_t$	Precipitation, evaporation and runoff over the combined lake (mm)
$A_M,\ A_H$	Surface areas of Lake Michigan and Lake Huron
$w_M,\ w_H$	Area weights of the two lakes
$Q_t^{\mathrm{Mic}},\ Q_t^{\mathrm{Hur}}$	Runoff flowing into each lake (m^3/s)
N_t	Number of days in month t
NBS_t	Net basin supply, $P_t - E_t + R_t$ (mm)
s_t	Rate at which the level rises during month t
ℓ	Level at the boundary between months $t-1$ and t

Before building any model, a quick check was done to see whether the bathtub picture holds (Figure 2). By Equation 6, the monthly level change should follow the mean NBS of this month and the previous month with a slope of 1. The fitted slope is 1.07 and the correlation is 0.81, so the picture holds well. The intercept is -69 mm per month. This is the water that leaves through the connecting rivers on average, which is not in the dataset. Spread over the lake surface, it corresponds to a net outflow of roughly $3{,}100\,\mathrm{m}^3/\mathrm{s}$.

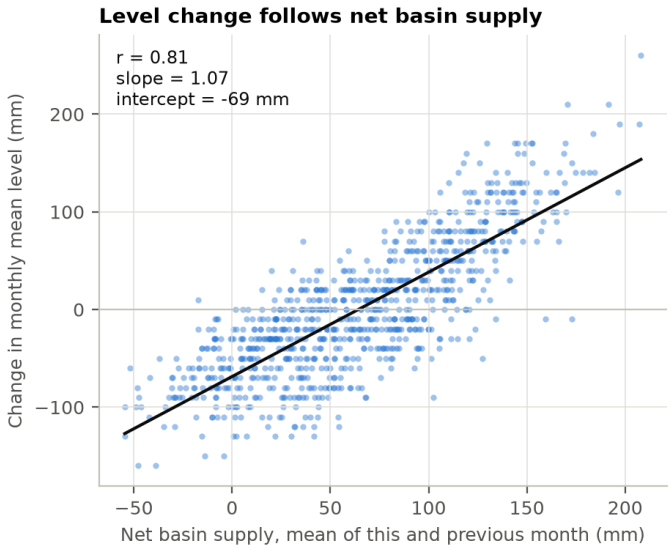


Figure 2: Monthly level change against net basin supply (mean of this month and the previous month), 1950–2023. The black line is a least-squares fit with a slope of 1.07 ($r = 0.81$).

2.3 Feature Engineering

Feature engineering was only needed for the gradient boosting model, since the physics-based models take their inputs straight from the table and the CNN reads the raw monthly series (Section 3.6). The idea was to give the gradient boosting model the same information that the water-balance model sees, plus a longer memory. A total of 18 features were chosen, and every one of them is known at the end of month t:

- **Current Level**: The level relative to a reference level, $L_t - L_{\text{ref}}$, with $L_{\text{ref}} = 176.5\,\text{m}$. The reference only centers the numbers and has no effect on the forecasts.
- **Recent Level Change**: The level change of the last three months, ΔL_t, ΔL_{t-1} and ΔL_{t-2}.
- **Precipitation, Evaporation and Runoff**: The values of P, E and R for the last three months, nine features in total.
- **Accumulated NBS**: The total supply over the last 3, 6 and 12 months,

$$S_t^{(w)} = \sum_{i=0}^{w-1} \text{NBS}_{t-i}, \qquad w \in \{3, 6, 12\}. \tag{7}$$

- **Season**: The calendar month m of month t, written as two numbers,

$$\sin\left(\frac{2\pi m}{12}\right) \quad \text{and} \quad \cos\left(\frac{2\pi m}{12}\right). \tag{8}$$

  It can be noticed that the month number by itself would be a poor feature, because it puts December (12) and January (1) eleven steps apart even though they are neighbors. Placing the months on a circle with a sine and a cosine fixes this: December and January end up next to each other, just like any other pair of months in a row.

Rolling-window features need a warm-up period before they are defined (for example, the 12-month sum needs 12 months of history), so the first 11 months of 1950 were not used to train this model. Table 3 summarizes the features.

Table 3: Features of the gradient boosting model at origin month t.

Group	Features	Count
Current level	$L_t - L_{\text{ref}}$	1
Recent level change	$\Delta L_t,\ \Delta L_{t-1},\ \Delta L_{t-2}$	3
Precipitation	$P_t,\ P_{t-1},\ P_{t-2}$	3
Evaporation	$E_t,\ E_{t-1},\ E_{t-2}$	3
Runoff	$R_t,\ R_{t-1},\ R_{t-2}$	3
Accumulated NBS	$S_t^{(3)},\ S_t^{(6)},\ S_t^{(12)}$	3
Season	$\sin(2\pi m/12),\ \cos(2\pi m/12)$	2
Total		18

2.4 Target Definition

To calculate the target, a simple question was asked. Standing at the end of month t, how much will the level change between now and h months from now? The month t is called the origin of the forecast and h is the horizon:

$$y_{t,h} = 1000\,(L_{t+h} - L_t)\,, \qquad h = 1, 2, \ldots, 6. \tag{9}$$

The forecast level is then $\hat{L}_{t+h} = L_t + \hat{y}_{t,h}/1000$. A forecast is only allowed to use the level and P, E, R up to and including month t.

The models predict the change in level, not the level itself. The level this month is almost identical to the level last month (their correlation is 0.987), so almost any model looks impressive if its predicted levels are simply compared with the observed levels. The month-to-month change is much harder to predict (its correlation with the previous month's change is 0.703), and it is the part a forecast actually has to get right.

2.5 Train/Test Split

As we are dealing with time series data, a randomly selected train/test split would not work. Neighboring months are strongly related and several features use rolling windows, so a random split would leak information from the test period into the training period. A single chronological split would avoid that, but it would test the models on only one stretch of years.

Instead, a rolling-origin backtest was used, which is also called evaluation on a rolling forecasting origin [8]. For each test year Y from 1980 to 2023, every model was trained again on all the months before January of year Y, and it then forecast every month of Y at every horizon. The forecast for target month T at horizon h is issued from the origin $T-h$. After that, the training window grows by one year and the process repeats. The first training set covers 1950–1979, so the first fit already has 30 years of data. This gives $44 \times 12 = 528$ forecasts per model and horizon, and every model was fitted without seeing any month of the year it was tested on.

3 Methodology

3.1 Baseline Models

Two simple models were built first. They are the benchmarks that any useful model has to beat.

Persistence. The simplest possible forecast is that the level stays where it is:

$$\hat{y}_{t,h} = 0. \tag{10}$$

Seasonal Climatology. The lake rises every spring and falls every autumn (Table 4). On average it rises by about 29 cm from March to July and falls by about the same amount from August to February. This model forecasts that each future month will change by the average change of that calendar month in the training years. If $\overline{\Delta L}_m$ is the average of ΔL over all training months that fall in calendar month m, and $m(t)$ is the calendar month of month t, then

$$\hat{y}_{t,h} = \sum_{k=1}^{h} \overline{\Delta L}_{m(t+k)}. \tag{11}$$

Since the seasonal cycle is so regular, this is a difficult benchmark to beat. A model that only learned the calendar would beat persistence easily, and that should not be mistaken for real forecasting skill.

Table 4: Average change in the monthly mean level for each calendar month, 1950–2023 (cm).

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Change	−3.7	−1.8	+1.5	+8.5	+9.8	+6.4	+3.1	−1.7	−5.1	−6.5	−5.5	−4.3

3.2 Water-Balance Model

The water-balance model is built on the bathtub idea. The volume of water in the lake, V, changes only through the water crossing its boundary:

$$\frac{dV}{dt} = A\,(p - e) + Q_R + Q_{\text{in}} - Q_{\text{out}}, \tag{12}$$

where A is the lake surface area, p and e are the precipitation and evaporation rates over the lake, Q_R is the runoff from the land, Q_{in} is the inflow from Lake Superior through the St. Marys River, and Q_{out} is the outflow through the St. Clair River. The area barely changes over the observed range of levels, so $dV = A\,dL$, and dividing by A turns every term into a depth of water:

$$\frac{dL}{dt} = p - e + \frac{Q_R}{A} + \frac{Q_{\text{in}} - Q_{\text{out}}}{A}. \tag{13}$$

The first three terms are exactly the NBS of Equation 4. The two river flows are not in the dataset, but their behavior is known in outline. The St. Clair River is not regulated, so its outflow grows when the lake is higher, roughly as $Q_{\text{out}} \approx Q_0 + k\,(L - L_{\text{ref}})$ with $k > 0$. A high lake therefore drains faster than a low one, and the level is always being pulled back toward an equilibrium.

This physics was turned into a linear regression:

$$\Delta L_t = a_{m(t)} + \sum_{c \in \{P,E,R\}} \left(b_{c,0}\, X_{c,t} + b_{c,1}\, X_{c,t-1}\right) + g\,(L_{t-1} - L_{\text{ref}}) + \varepsilon_t, \tag{14}$$

where $X_{P,t}$, $X_{E,t}$ and $X_{R,t}$ are P_t, E_t and R_t. Every part of it has a meaning:

- The two months of every supply term come from the monthly means (Equation 6).
- The twelve monthly constants a_m stand in for the average flow through the connecting rivers, which changes through the year.
- The level term $g\,(L_{t-1} - L_{\text{ref}})$ is the outflow that grows with the level, so g should be negative.
- For a perfect water budget, $b_{P,0} + b_{P,1} = 1$, $b_{E,0} + b_{E,1} = -1$ and $b_{R,0} + b_{R,1} = 1$.

That is $12 + 6 + 1 = 19$ numbers to learn. They were found by ordinary least squares, which picks the coefficients that make the sum of squared errors as small as possible:

$$F(\beta) = \sum_t \left(\Delta L_t - \widehat{\Delta L_t}\right)^2, \tag{15}$$

where β collects all 19 coefficients and $\widehat{\Delta L_t}$ is the value of Equation 14 without the error term ε_t.

Forecasting with Unknown Weather. To forecast, the future P, E and R are needed, but they are unknown. Each one was replaced by its calendar-month average from the training years, plus a fading share of how unusual the latest month was:

$$\hat{X}_{c,t+k} = \overline{X}_{c,m(t+k)} + \phi_c^{\,k}\left(X_{c,t} - \overline{X}_{c,m(t)}\right), \tag{16}$$

where ϕ_c is the correlation between one month's anomaly and the next month's anomaly in the training data. This is called a first-order autoregression, or AR(1) [8]. The model is then stepped forward one month at a time, feeding its own predicted level back in:

$$\hat{L}_{t+k} = \hat{L}_{t+k-1} + \frac{\widehat{\Delta L_{t+k}}}{1000}, \qquad \hat{L}_t = L_t. \tag{17}$$

Perfect Forcing. The same model was also run with the observed future P, E and R in place of Equation 16. This uses information from the future, so it is not a real forecast. It shows how good the model could be with a perfect weather forecast, which tells us how much of the error comes from not knowing the weather.

Table 5 summarizes the notation used in Equations 12–17.

3.3 Lake Model

The water-balance model turns Equation 13 into a linear regression on monthly totals. The lake model takes a more direct route: it solves Equation 13 forward in time, with the outflow written in the way water leaves a lake over a natural control. Flow over a weir grows with the depth of water above its crest to the power 3/2, so the outflow was taken to be proportional to $(L - z_0)^{3/2}$, where z_0 is the elevation of an effective crest. With every flow written in mm per month over the lake, the model is

$$1000\,\frac{dL}{dt} = m_P\,P_t - m_E\,E_t + m_R\,R_t + n_{m(t)} - c\left[\left(\frac{L - z_0}{d}\right)^{3/2} - 1\right], \qquad z_0 = L_{\text{ref}} - d, \tag{18}$$

where L is in meters, time is in months, and P_t, E_t and R_t are the supply terms of the month that the lake is in. Every part of it has a meaning:

Table 5: Notation used in the water-balance model.

Symbol	Meaning
V	Volume of water in the lake
A	Surface area of the combined lake
$p,\ e$	Precipitation and evaporation rates over the lake
Q_R	Runoff from the land basin
$Q_{\text{in}},\ Q_{\text{out}}$	Inflow from Lake Superior and outflow through the St. Clair River
$Q_0,\ k$	Outflow at the reference level, and how fast the outflow grows with the level
$m(t)$	Calendar month (1 to 12) of month t
$X_{c,t}$	Supply term c (precipitation, evaporation or runoff) in month t (mm)
a_m	Constant for calendar month m (mm)
$b_{c,0},\ b_{c,1}$	Weights of supply term c in this month and in the previous month
g	Level coefficient (mm per month per meter of level)
L_{ref}	Reference level, 176.5 m
ε_t	Error term
β	All 19 coefficients of the model
$\overline{X}_{c,m}$	Average of supply term c in calendar month m over the training years
ϕ_c	Month-to-month correlation of the anomalies of supply term c
$\hat{X}_{c,t+k},\ \hat{L}_{t+k}$	Forecast supply and forecast level k months after the origin

- **Scale Factors**: m_P, m_E and m_R scale the supply terms, which are themselves estimates with their own errors. For a closed water budget all three would be 1.
- **Channel Flow**: n_m is the net flow through the connecting rivers when the lake is at the reference level, one value for each calendar month, because these flows have a seasonal cycle.
- **Outflow**: The last term is the extra outflow when the lake is above the reference level, and it is negative when the lake is below it. In the model, c is the outflow at the reference level and d is the depth of water above the crest. Near the reference level the outflow grows by $\frac{3}{2}\,c/d$ mm per month for every meter of level, which plays the role of $-g$ in Equation 14.

Monthly Means. Equation 18 was stepped forward ten times per month. If $f(L)$ is its right-hand side, each step is

$$L^{(j+1)} = L^{(j)} + \frac{f\big(L^{(j)}\big)}{1000 \times 10}, \tag{19}$$

and the forecast for month $t+k$ is the average of the ten levels in that month, $\bar{L}_{t+k}$, because the observed levels are monthly means. This builds Equation 6 into the simulation instead of estimating it. A run starts from the observed mean level of the origin month, which belongs to the middle of that month, and first takes one half-month step with that month's supply to reach its end, $L^{(0)} = L_t + f(L_t)/2000$. The forecast is then $\hat{y}_{t,h} = 1000\,(\bar{L}_{t+h} - L_t)$.

Fitting. The 17 numbers $\theta = (m_P, m_E, m_R, n_1, \ldots, n_{12}, c, d)$ were fitted by non-linear least squares to the observed level changes at all six horizons, with the observed supply as input:

$$F(\theta) = \sum_{t} \sum_{h=1}^{6} \left(y_{t,h} - \hat{y}_{t,h}(\theta)\right)^2 . \tag{20}$$

This was done with the `least_squares` function of SciPy [9], a trust-region method that keeps every number inside given bounds. The scale factors were kept between 0 and 3, c above 0, and d between 1.5 and 50 m. The lower bound on d keeps the crest at or below 175.0 m, under the record low of 175.57 m, so that $L - z_0$ stays positive. The search started from $m_P = m_E = m_R = 1$, from n_m equal to the average of $\Delta L_t - \text{NBS}_t$ in calendar month m, and from $c = 80$ mm per month and $d = 5$ m, so that the outflow slope $\frac{3}{2}\,c/d = 24$ mm per month per meter started close to the level coefficient of the water-balance model. Fitting all six horizons

at once, instead of one month, makes the numbers describe how the lake behaves over the whole forecast range.

Forecasting. The future P, E and R were filled in exactly as for the water-balance model (Equation 16), and a perfect-forcing run used the observed values instead. Table 6 summarizes the notation used in Equations 18–20.

Table 6: Notation used in the lake model.

Symbol	Meaning
$m_P,\ m_E,\ m_R$	Scale factors of precipitation, evaporation and runoff
n_m	Net channel flow at the reference level in calendar month m (mm per month)
c	Outflow at the reference level (mm per month)
d	Depth of water above the effective crest at the reference level (m)
z_0	Elevation of the effective crest, $L_{\mathrm{ref}} - d$ (m)
$f(L)$	Right-hand side of Equation 18 (mm per month)
$L^{(j)}$	Level after step j of the simulation (m)
$\bar{L}_{t+k}$	Average of the simulated levels in month $t+k$ (m)
θ	All 17 numbers of the lake model

3.4 Gradient Boosting

The machine learning model of choice here was gradient boosting [10] with regression trees [11], from the scikit-learn library [12]. A regression tree splits the data with a series of yes-or-no questions about the features (for example, *was the runoff this month above 40 mm?*) and predicts the average target of each final group, called a leaf. One small tree is a weak model on its own, but gradient boosting adds many of them together, and each new tree corrects the mistakes left by the trees before it:

$$F_M(\mathbf{x}) = F_0 + \nu \sum_{j=1}^{M} T_j(\mathbf{x}), \tag{21}$$

where F_0 is the average of the training targets. Each tree T_j is fitted to the residuals of the model built so far,

$$r_i^{(j)} = y_i - F_{j-1}(\mathbf{x}_i), \tag{22}$$

which is the direction that reduces the squared error the fastest. The learning rate ν shrinks the step of each tree, so the model learns slowly and is less likely to overfit.

A separate model was trained for each horizon $h = 1, \ldots, 6$, and each one predicts $y_{t,h}$ directly from the 18 features in Table 3. The settings were fixed in advance and were not tuned on the test years (Table 7). Because each tree is fitted to a random part of the training rows, this version is called stochastic gradient boosting [13].

Table 7: Settings of the gradient boosting model.

Setting	Value	Meaning
Number of trees, M	300	Trees added together in each model
Learning rate, ν	0.03	Shrinks the contribution of each tree
Maximum depth	3	Each tree asks at most three questions in a row
Subsample	0.8	Each tree is fitted to a random 80% of the training rows
Minimum leaf size	5	Every leaf holds at least 5 months
Random seed	0	Makes the results repeatable

3.5 Hybrid Model

The hybrid model combines the two approaches. The trees are not trained on the level change itself, but on what the water-balance model got wrong:

$$\hat{y}_{t,h}^{\text{hyb}} = \hat{y}_{t,h}^{\text{WB}} + F_M^{(h)}(\mathbf{x}_t), \qquad F_M^{(h)} \text{ trained on } y_{t,h} - \hat{y}_{t,h}^{\text{WB}}. \tag{23}$$

This is the most direct test of whether machine learning can find a pattern that the physics-based model misses. If the water-balance model leaves anything systematic behind, the trees should find it.

Table 8 summarizes the additional notation used in Equations 21–23.

Table 8: Additional notation used in the gradient boosting and hybrid models.

Symbol	Meaning
$\mathbf{x}_t$	The 18 features at origin month t
y_i	Target of training row i
F_M	Gradient boosting model made of M trees
F_0	Average of the training targets
T_j	The j-th regression tree
ν	Learning rate
$r_i^{(j)}$	Residual of training row i that tree j is fitted to
$\hat{y}_{t,h}^{\text{WB}}$	Water-balance forecast of the level change
$\hat{y}_{t,h}^{\text{hyb}}$	Hybrid forecast of the level change
$F_M^{(h)}$	Trees of the hybrid for horizon h, trained on the water-balance errors

3.6 Physics-Guided Convolutional Neural Network

The last model asks whether a neural network does better when it is made to respect the physics. Instead of predicting the level, the network is only allowed to change the one input that the physics cannot know: the weather of the coming months. The lake model of Section 3.3 then turns that weather into levels, so every forecast obeys the water balance. This follows the idea of process-guided deep learning, in which knowledge of the physical process is built into how a neural network is trained [4].

Input. For each origin month t, the network sees the last 24 months of seven series: P, E, R, ΔL, L, and the sine and cosine of the month (Equation 8). Each series is standardized with its mean and standard deviation in the training years, so the input is a 7×24 table of numbers.

Network. A convolutional neural network (CNN) [14] slides small filters along the time axis, so the same pattern, for example a wet month followed by high runoff, is recognized wherever it occurs in the 24 months. Two convolution layers with eight filters of width three, each followed by the activation $\max(0, z)$, feed one linear layer with 18 outputs $\delta_{c,k}$, one for each supply term c and each month $k = 1, \ldots, 6$ ahead. They correct the AR(1) weather forecast of Equation 16:

$$\tilde{X}_{c,t+k} = \hat{X}_{c,t+k} + \sigma_c \, \delta_{c,k}, \tag{24}$$

where σ_c is the standard deviation of the anomalies of supply term c in the training years. The weights of the last layer start at zero, so before any training the network gives exactly the forecast of the lake model.

Training Through the Physics. The 17 numbers of the lake model were fitted first and then held fixed. Equations 18 and 19 use only arithmetic, so the error of the final level forecast can be passed back through the simulated lake to the weights of the network. This was done with PyTorch [15]. The loss has two parts,

$$J = \frac{1}{6N} \sum_t \sum_{h=1}^{6} \left(\frac{\hat{y}_{t,h} - y_{t,h}}{s_h} \right)^2 + \frac{1}{18N} \sum_t \sum_c \sum_{k=1}^{6} \left(\frac{\tilde{X}_{c,t+k} - X_{c,t+k}}{\sigma_c} \right)^2, \tag{25}$$

where N is the number of training origins and s_h is the standard deviation of $y_{t,h}$ in the training years. The first part is the error of the level forecast. The second part is the error of the corrected weather, which keeps the corrected precipitation, evaporation and runoff believable instead of bent to fit the levels.

Early Stopping. With only a few hundred training months, a neural network can easily memorize noise. The network was therefore trained on the first 80% of the training origins and checked after every pass through them (an epoch) on the last 20%, and the epoch with the lowest loss on those held-out months was kept [16]. If no epoch beat the untrained network, the CNN stays identical to the lake model. The weights were updated with the AdamW method [17], which shrinks the weights a little at every step (weight decay) to limit overfitting.

The settings in Table 9 were chosen with data from 1950–1979 only, by training on earlier years and checking the forecasts for the 1970s. Designs without early stopping were rejected at this stage because they overfit: their error on the held-out years grew as training went on. The test years were never used.

Table 9: Settings of the physics-guided CNN.

Setting	Value	Meaning
Input window	24 months	History the network sees at each origin
Input series	7	P, E, R, ΔL, L and the sine and cosine of the month
Convolution layers	2	Each with 8 filters of width 3
Outputs	18	Corrections to P, E and R for each of the next 6 months
Learning rate	0.001	Size of each update of the weights
Weight decay	0.01	Shrinks the weights at every step
Maximum epochs	300	Passes through the training origins
Held-out share	20%	Last training origins, used for early stopping
Random seed	0	Makes the results repeatable

Table 10 summarizes the additional notation used in Equations 24 and 25.

Table 10: Additional notation used in the physics-guided CNN.

Symbol	Meaning
$\delta_{c,k}$	Output of the network: correction to supply term c, k months ahead
$\tilde{X}_{c,t+k}$	Corrected forecast of supply term c
σ_c	Standard deviation of the anomalies of supply term c
J	Loss minimized in training
N	Number of training origins
s_h	Standard deviation of the target $y_{t,h}$ in the training years

3.7 Evaluation

The accuracy of each model is reported as the root-mean-square error (RMSE) of the forecast level, in cm, separately for each horizon:

$$\mathrm{RMSE}_h = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\hat{L}_{i,h} - L_i\right)^2}, \tag{26}$$

where L_i is the observed level in target month i, $\hat{L}_{i,h}$ is its forecast made h months earlier, and the sum runs over the $n = 528$ target months. The improvement of a model over a reference model is the relative reduction in RMSE,

$$\mathrm{Skill}_h = 1 - \frac{\mathrm{RMSE}_h^{\mathrm{model}}}{\mathrm{RMSE}_h^{\mathrm{ref}}}. \tag{27}$$

A small difference in RMSE between two models could just be luck. To check whether a difference is real, the 44 test years were resampled with replacement 2,000 times, which is called a bootstrap [18]. For each resample, the RMSE difference between the two models was computed again, and the middle 95% of the 2,000 differences gives a 95% interval. Whole years were resampled together, because the errors within a year, and across overlapping multi-month forecasts, are strongly related to each other. Resampling whole blocks of neighboring observations instead of single values is known as a block bootstrap [19].

To see which inputs the gradient boosting model actually uses, its permutation importance was measured [20]. For this, the model was trained on 1950–2009 and scored on 2010–2023. One group of features G was shuffled at a time, which destroys the information in that group while keeping its values realistic, and the increase in RMSE was recorded:

$$I_G = \frac{1}{20}\sum_{q=1}^{20} \mathrm{RMSE}_q\left(G \text{ shuffled}\right) - \mathrm{RMSE}\left(\text{nothing shuffled}\right). \tag{28}$$

Table 11 explains each of these metrics.

Table 11: Explanation of the metrics used to evaluate the models.

Metric	Description
RMSE	The typical size of a forecast miss, in cm. Errors are squared before they are averaged, so large misses count more than small ones. Lower is better.
Skill	The fraction by which a model reduces the RMSE of a reference model. 0% means no improvement and 100% means a perfect forecast.
95% interval	The range that holds the middle 95% of the 2,000 bootstrap differences between two models. If it includes zero, the two models cannot be told apart.
Permutation importance	How much the RMSE increases when one group of features is shuffled. The larger the increase, the more the model depends on that group.

4 Results

4.1 Accuracy by Horizon

Table 12 and Figure 3 show the RMSE of every model at every horizon.

Table 12: RMSE of the forecast level (cm) by horizon h in months, test years 1980–2023. Bold marks the best real forecast at each horizon. The last two rows use the observed future weather and are not real forecasts.

Model	$h = 1$	$h = 2$	$h = 3$	$h = 4$	$h = 5$	$h = 6$
Persistence	6.73	12.37	17.06	20.73	23.30	24.81
Seasonal climatology	4.14	7.06	9.48	11.63	13.53	15.27
Water balance	**3.22**	**6.33**	**8.82**	**11.03**	**12.96**	14.66
Lake model	3.26	6.38	8.88	11.12	13.08	14.78
Gradient boosting	3.32	6.51	9.12	11.40	13.48	15.39
Hybrid (water balance + boosting)	3.26	6.35	8.95	11.18	13.00	**14.58**
Physics-guided CNN	3.26	6.37	8.87	11.10	13.05	14.76
Water balance, perfect forcing	2.39	3.77	5.08	6.29	7.43	8.49
Lake model, perfect forcing	2.51	3.93	5.29	6.55	7.73	8.83

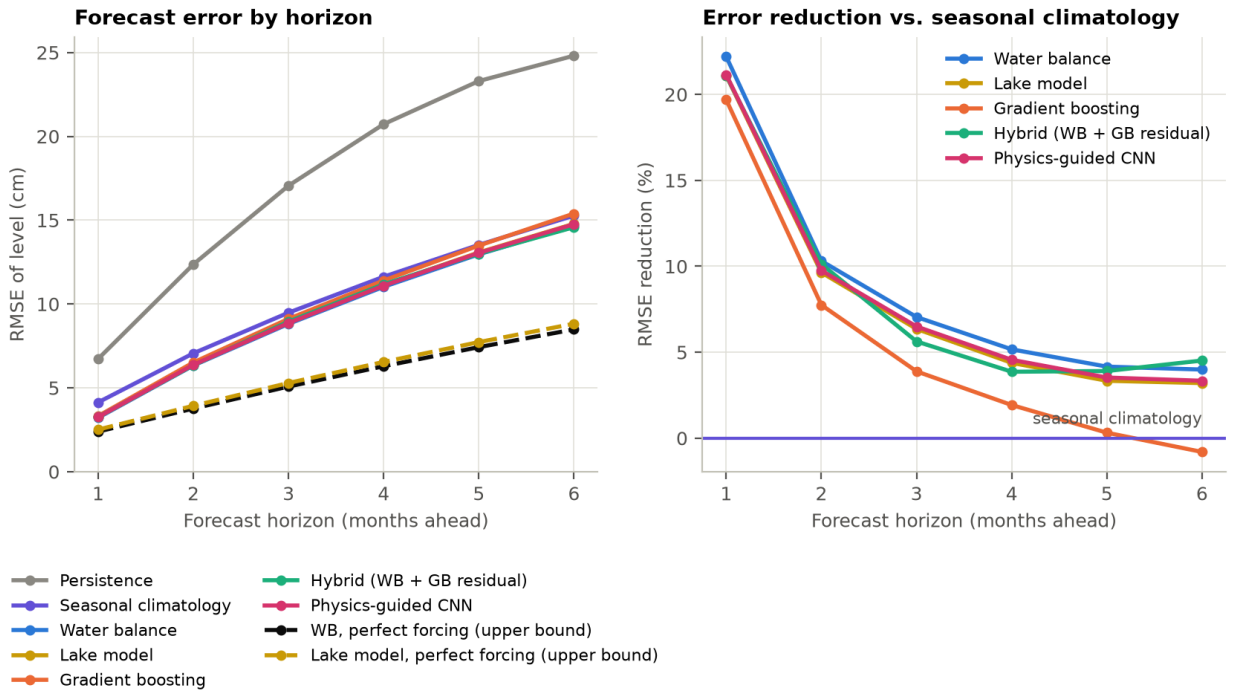


Figure 3: Left: RMSE of the forecast level by horizon for every model; the dashed lines use the observed future weather and are not real forecasts. Right: reduction in RMSE relative to the seasonal climatology.

The main results are:

- **Every Model Beats Persistence**: The water-balance model reduces the RMSE of persistence by 52% at one month and 41% at six months. But the seasonal climatology alone already reduces it by 38–44%, so most of that improvement comes from simply knowing the time of year.
- **Skill Beyond the Season Fades Quickly**: Compared with the seasonal climatology, the water-balance model reduces the RMSE by 22% at one month, 10% at two months, 7% at three months and only 4–5% at four to six months.
- **Gradient Boosting Did Not Win**: It had a slightly higher RMSE than the water-balance model at every horizon, and at five and six months it was no better than the seasonal climatology.
- **The Hybrid Did Not Help**: Letting the trees correct the water-balance errors changed the RMSE by only -0.08 to $+0.15$ cm.
- **More Physical Detail Did Not Help**: The lake model was slightly worse than the water-balance model at every horizon, by 0.05 cm at one month and 0.12 cm at six months.
- **The CNN Stayed Close to the Lake Model**: Its RMSE was slightly lower than that of the lake model at every horizon, but by at most 0.03 cm (Section 4.5).
- **The Weather Sets the Limit**: With the observed future weather, the same water-balance model reached 2.39 cm at one month and 8.49 cm at six months. That is 26% lower than the real water-balance forecast at one month and 40–43% lower at two to six months. The lake model with the observed weather improved on its own forecasts by a similar 23% and 38–41%. The gap between these dashed lines and the real forecasts in Figure 3 is far larger than any gap between the models.

4.2 Is the Difference Real?

Table 13 shows the bootstrap comparison for the four most important pairs of models from the first question, and Table 14 shows it for the two new models.

Table 13: Difference in RMSE (cm) between two models, with the 95% bootstrap interval in brackets. GB is gradient boosting and WB is the water-balance model. A negative value means the first model is better.

Horizon	GB − WB	Hybrid − WB	WB − Climatology	GB − Climatology
1	+0.10 [−0.02, 0.23]	+0.05 [−0.04, 0.13]	−0.92 [−1.12, −0.70]	−0.81 [−1.05, −0.58]
2	+0.18 [−0.07, 0.44]	+0.01 [−0.17, 0.19]	−0.73 [−1.17, −0.28]	−0.55 [−0.97, −0.13]
3	+0.30 [−0.09, 0.68]	+0.13 [−0.18, 0.45]	−0.67 [−1.26, −0.08]	−0.37 [−0.95, 0.21]
4	+0.38 [−0.15, 0.92]	+0.15 [−0.29, 0.62]	−0.60 [−1.41, 0.20]	−0.22 [−0.96, 0.48]
5	+0.52 [−0.16, 1.21]	+0.03 [−0.53, 0.62]	−0.56 [−1.59, 0.48]	−0.04 [−0.90, 0.82]
6	+0.73 [−0.14, 1.63]	−0.08 [−0.78, 0.65]	−0.61 [−1.99, 0.65]	+0.12 [−1.01, 1.20]

The gap between gradient boosting and the water-balance model grows from 0.10 cm at one month to 0.73 cm at six months, but every one of its intervals includes zero. So gradient boosting is no better than the water-balance model, and possibly slightly worse. Every interval for the hybrid also includes zero. The water-balance model is clearly better than the seasonal climatology at one to three months, where the whole interval is below zero, but not at four to six months. Gradient boosting is clearly better than the seasonal climatology only at one and two months.

Table 14: Difference in RMSE (cm) for the two new models, with the 95% bootstrap interval in brackets. WB is the water-balance model. A negative value means the first model is better.

Horizon	Lake model − WB	CNN − Lake model
1	+0.05 [0.02, 0.07]	−0.00 [−0.01, 0.01]
2	+0.05 [0.00, 0.09]	−0.01 [−0.03, 0.01]
3	+0.07 [0.00, 0.13]	−0.01 [−0.06, 0.02]
4	+0.09 [0.01, 0.17]	−0.02 [−0.11, 0.06]
5	+0.11 [0.01, 0.21]	−0.03 [−0.15, 0.10]
6	+0.12 [−0.00, 0.24]	−0.02 [−0.19, 0.14]

The lake model is worse than the water-balance model by only 0.05–0.12 cm, but the gap is so steady from year to year that its interval stays at or above zero at one to five months and only just reaches zero at six months. So the extra physical detail made the forecasts slightly, but measurably, worse. The CNN and the lake model cannot be told apart at any horizon.

4.3 What the Water-Balance Model Learned

Table 15 lists the coefficients of Equation 14 fitted on the full record.

Table 15: Water-balance weights of the supply terms fitted on 1950–2023, and the month-to-month correlation ϕ_c of each supply term. The weights have no units (mm of level change per mm of supply).

Supply term	$b_{c,0}$	$b_{c,1}$	Sum	Expected	ϕ_c
Precipitation	0.35	0.53	0.88	+1	0.02
Evaporation	−0.43	−0.37	−0.81	−1	0.26
Runoff	1.22	0.61	1.83	+1	0.45

The coefficients make physical sense in sign. Both months of every supply term carry weight, just as Equation 6 predicts. The sums for precipitation (0.88) and evaporation (−0.81) are reasonably close to the ±1 expected from the water budget. The twelve monthly constants a_m are all negative, from −136 mm per

month in October to −67 mm per month in May, which mostly reflects the net loss of water through the connecting rivers.

The level coefficient is $g = -23$ mm per month per meter of level. It is negative, which matches an outflow that grows as the lake rises. Its size means that an unusually high or low level fades with a time scale of

$$\tau = \frac{1000 \text{ mm per meter}}{23 \text{ mm per month per meter}} \approx 43 \text{ months} \approx 3.6 \text{ years}. \tag{29}$$

This is a value implied by the regression, not a measurement of the river. It does show why the lake has a memory of several years: an anomaly in the level fades over years, not months.

The runoff weights sum to 1.83. Runoff should not raise the lake by more than its own volume, so runoff must be standing in for something that is not measured. Two likely candidates are the inflow from Lake Superior, which tends to be high in the same wet periods, and runoff from parts of the basin without river gauges. The model should therefore be read as shaped by physics, not as a closed water budget.

The supply terms are barely predictable from their own past: ϕ_c is 0.02 for precipitation, 0.26 for evaporation and 0.45 for runoff. In Equation 16 the anomaly is multiplied by ϕ_c^k, so even for runoff only $0.45^3 \approx 9\%$ of an anomaly is left after three months. This is why the skill fades so quickly: after a month or two, the best guess for the future supply is just the seasonal average.

4.4 What the Lake Model Learned

Table 16 lists the main numbers of the lake model fitted on the full record.

Table 16: Main numbers of the lake model (Equation 18) fitted on 1950–2023.

Number	Symbol	Fitted value	Closed water budget
Precipitation scale	m_P	1.00	1
Evaporation scale	m_E	0.86	1
Runoff scale	m_R	1.73	1
Outflow at the reference level	c	24.72 mm per month	–
Depth above the crest	d	1.50 m (lower bound)	–
Outflow slope near L_{ref}	$\frac{3}{2}\,c/d$	24.7 mm per month per meter	–

Precipitation is used at its estimated value and evaporation at 86% of it. Runoff is again inflated, by a factor of 1.73, close to the 1.83 found by the regression (Table 15). The more physical structure therefore did not close the gap in the water budget: runoff still stands in for something that is not measured. All twelve channel terms n_m are negative, as expected for a lake that loses water through its rivers on average.

The outflow slope near the reference level, 24.7 mm per month per meter, is almost the same as the 23 mm per month per meter of the regression. The depth d, on the other hand, ended on its lower bound of 1.5 m, so the data asked for an outflow that bends with the level more sharply than the bound allows. The size of the outflow is not pinned down either. The value of c corresponds to only about 1,100 m^3/s, less than even the average net outflow of about 3,100 m^3/s found in the check of Figure 2. Without measured river flows, c and n_m can only be fitted together, so the model learns how strongly the outflow responds to the level, but not how large the outflow really is.

4.5 What the CNN Learned

Early stopping kept the untrained network, which is exactly the lake model, in 39 of the 44 test years. Only in the fits for 2001, 2002, 2003, 2004 and 2007 did some epoch beat it on the held-out training months, and even then the overall RMSE moved by at most 0.03 cm. The same thing was seen while the settings were chosen on 1950–1979: the error on the training months kept falling with every epoch, while the error on the held-out months rose from the first epoch on. The network could fit the training years, but it found nothing that carried over to new years. This agrees with the short memory of the supply terms (Table 15): beyond

what the AR(1) forecast already uses, the last 24 months of the lake's own record say little about the coming weather.

4.6 What the Machine Learning Model Relies On

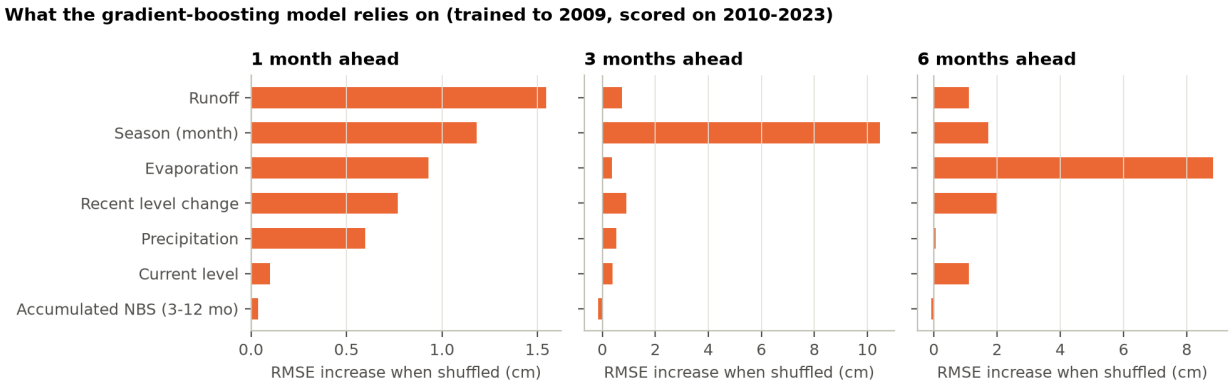


Figure 4: Permutation importance for the gradient boosting model, trained on 1950–2009 and scored on 2010–2023. Each bar shows how much the RMSE increases when one group of features is shuffled.

At one month ahead, the model relies on recent runoff, the season, evaporation, the latest level change and precipitation, in that order (Figure 4). This matches the physics: by Equation 6, last month's supply is still working its way into this month's mean level. At three months the model is dominated by the season. At six months the largest group is evaporation and not the season, but evaporation has such a strong seasonal cycle (Figure 1) that the model is most likely using it as another way of reading the time of year. Permutation importance cannot separate features that carry the same information, so the order among the seasonal features should not be over-interpreted. The clear finding is that at longer horizons the model has learned the seasonal cycle and little else.

4.7 Case Study: The 2013 Low to the 2020 High

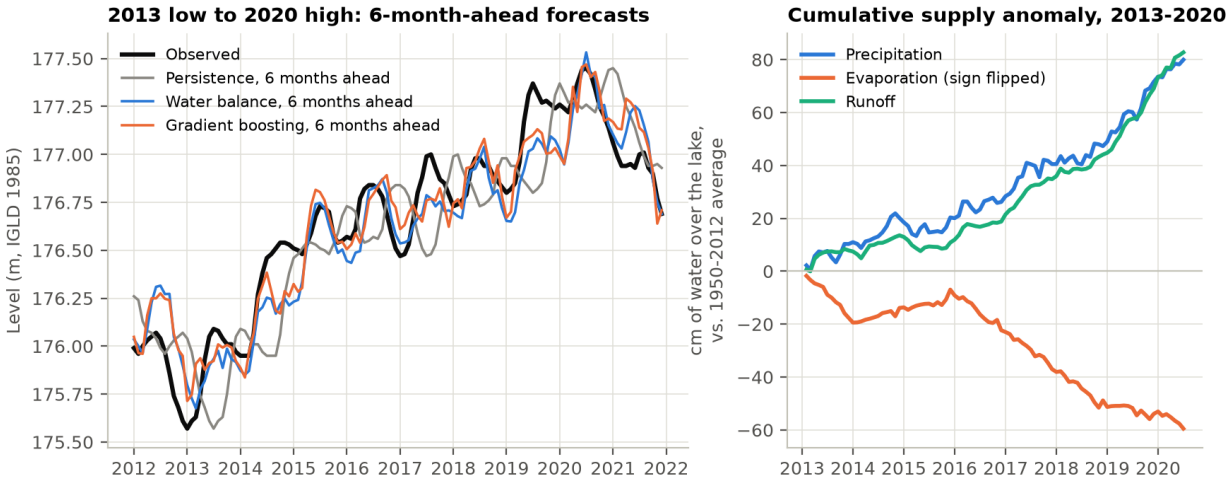


Figure 5: Left: observed level and six-month-ahead forecasts, 2012–2021. Right: cumulative anomalies of the supply terms from February 2013 to July 2020, relative to the 1950–2012 monthly averages. Evaporation is plotted with its sign flipped, so that upward always means more water in the lake.

Between January 2013 and July 2020 the lake rose 1.88 m. Compared with the 1950–2012 averages, the precipitation over the lake delivered about 80 cm of extra water during this period and the runoff about 83 cm (Figure 5, right). Evaporation was below average in 2014 and 2015, which Gronewold et al. [1] also found for 2014, but over the whole period it worked in the opposite direction: it was above average and removed about 59 cm more water than usual. So the rise was driven by a long wet period, not by lower evaporation, which agrees with the high precipitation and river flows reported for these years [1, 2]. The net supply anomaly, about +1.0 m, is smaller than the observed rise, and the rest cannot be explained with this dataset. Likely contributors are the normal seasonal rise from January to July (about 27 cm on average), lower outflow while the lake was low, and the inflow from Lake Superior, which was very high in 2014 [1].

Both the water-balance model and gradient boosting followed the rise at a six-month lead with similar accuracy, with an RMSE of 15.0 and 14.6 cm for target months in 2013–2020, against 26.7 cm for persistence. The lake model and the CNN were close behind, at 15.1 cm each. None of them saw the rise coming. Figure 5 shows the water-balance model and gradient boosting lagging behind the two steepest climbs, in 2014 and 2019. This is exactly what is expected from a model that fills in the future supply with averages: it can follow a rise step by step as the wet months arrive, but it cannot know in advance that the next six months will be wet again.

Given the observed weather, however, the lake model did better than the water-balance model in these years at every horizon, with 8.41 against 8.85 cm at six months, even though it was worse over the whole test period (8.83 against 8.49 cm). The years 2013–2020 run from the record low to close to the record high, where a curved outflow and a straight one differ the most, which is the likely reason. This is the one place where the extra physical detail showed an advantage, and it appears only when the weather is known.

5 Discussion

The results of this case study point to one main finding: no model clearly beat the simple water-balance regression. More flexibility (gradient boosting, the hybrid and the CNN) did not improve on it, and neither did more physical detail (the lake model). The reason can be seen in the physics. Equation 13 is linear in the supply terms, so a linear regression with the right inputs already has the correct shape. The trees have to rediscover that shape from only a few hundred training months, and they have nothing to gain from being non-linear. The hybrid model shows the same thing from the other side: when the trees were asked only for what the water-balance model missed, they found nothing systematic.

The lake model shows that more physical detail is not automatically better. Its curved outflow and its direct treatment of monthly means are closer to how the lake works, yet it was slightly worse than the regression. The largest gap in the physics is probably not the shape of the outflow but the river flows that are not measured. Both models have to absorb the inflow from Lake Superior into other terms, and both inflate runoff to do it (by 1.73 and 1.83). The regression, with two free weights for every supply term, has more room to absorb that gap than the lake model, which has one scale factor for each. The lake model's one clear advantage, in 2013–2020 with the observed weather, suggests that its structure would pay off once the inflow is measured.

The physics-guided CNN asked whether a neural network could forecast the coming weather better than the AR(1) model from the recent record of the lake. It could not: in 39 of the 44 test years, early stopping kept the untrained network. The design did have one benefit. Because the network starts as the lake model and early stopping only accepts changes that help on held-out years, it could not do much harm, and its error stayed within 0.03 cm of the lake model at every horizon.

The perfect-forcing runs show where the real limit is. Knowing the future weather would cut the six-month error of the water-balance model from 14.66 cm to 8.49 cm, a 42% reduction, which is far more than any change of model achieved. The lake model gains almost as much, from 14.78 cm to 8.83 cm. Precipitation has almost no memory from one month to the next ($\phi_P = 0.02$), so no model that only looks at past data can know whether the coming months will be wet.

This study also has some limitations:

1. **Level Data End in 2023**: The level file runs to October 2025 but has no values after December 2023, so 2024 could not be tested.
2. **Early Evaporation Data**: Lake Michigan evaporation for 1941–1944 appears to be filled in, so the analysis starts in 1950.
3. **Changing Runoff Coverage**: The share of each basin with river gauges changes over time (Michigan 0.78–0.98, Huron 0.74–0.93 during 1950–2023). It is kept in the table as a data-quality column but is not used as a feature. Coverage drops sharply in 2024, when Huron is only about 60% gauged, and runoff is missing after August 2024. This is outside the analysis period, but it matters for any extension.
4. **No Connecting-River Flows**: The St. Marys inflow, the St. Clair outflow and the Chicago diversion are not in the dataset. The water-balance model represents them only through the monthly constants and the level term, and the lake model through its channel terms and outflow curve. In both, the inflated runoff weight suggests that other terms are absorbing part of this gap, and in the lake model the depth d stopped at its lower bound.
5. **Slow Drift**: Evaporation averaged 37–40 mm per month in the 1970s–1990s and 51–52 mm per month since 2010. On average the forecasts are slightly too high, by about 1–2 cm at six months for the real forecasts and 4.8–4.9 cm for the two perfect-forcing runs. This is a sign of slow changes that a model trained on earlier decades does not fully capture.
6. **Level Resolution**: Levels are reported to the nearest centimeter, which adds a small amount of rounding noise to the monthly changes.
7. **Few Machine Learning Models**: Only two algorithms were tested, gradient boosting and a small CNN, each with one fixed setting and on a few hundred training months. The result shows that a reasonable off-the-shelf model, and a small network built around the physics, do not beat the water-balance model here. It does not show that no machine learning approach could.
8. **Data Delay**: P, E and R for month t are assumed to be known at the end of month t. In practice these estimates are published later, so the real-time skill would be somewhat lower.

6 Conclusion

This case study built a complete forecasting pipeline for the monthly level of Lake Michigan–Huron and compared machine learning models with models built on the physics of the lake. No model was clearly more accurate than a 19-parameter water-balance regression at any horizon from one to six months: not gradient boosting, not a hybrid of the two, not a more detailed 17-parameter lake model with a curved outflow, and not a CNN that corrects the weather driving that lake model. Most of the improvement over persistence came from the seasonal cycle. Knowing the recent supply added a further 22% at one month and little beyond three months. The run with the observed weather showed that the error could be cut by more than 40% at two to six months if the future precipitation, evaporation and runoff were known.

The practical lesson is that the next improvement is more likely to come from better inputs than from a more flexible or a more detailed model. Useful inputs would be seasonal climate forecasts of precipitation and temperature, the snowpack at the end of winter (which controls the spring runoff), the lake surface temperature and ice cover (which control evaporation), and measured flows in the connecting rivers, which would also let the lake model make use of its physical structure. Machine learning may well earn its place once those inputs are available, and this backtest gives the baseline to test it against.

References

[1] A. D. Gronewold, J. Bruxer, D. Durnford, et al. Hydrological drivers of record-setting water level rise on Earth's largest lake system. *Water Resources Research*, 52(5):4026–4042, 2016. `https://doi.org/10.1002/2015WR018209`

[2] A. D. Gronewold and R. B. Rood. Recent water level changes across Earth's largest lake system and implications for future variability. *Journal of Great Lakes Research*, 45(1):1–3, 2019. https://doi.org/10.1016/j.jglr.2018.10.012

[3] U.S. Environmental Protection Agency. Physical features of the Great Lakes. https://www.epa.gov/greatlakes/physical-features-great-lakes (accessed October 19, 2026).

[4] J. S. Read, X. Jia, J. Willard, et al. Process-guided deep learning predictions of lake water temperature. *Water Resources Research*, 55(11):9173–9190, 2019. https://doi.org/10.1029/2019WR024922

[5] NOAA Great Lakes Environmental Research Laboratory. Great Lakes monthly hydrologic data (1860–2024). https://www.glerl.noaa.gov/ahps/mnth-hydro.html. Data files: https://www.glerl.noaa.gov/pubs/tech_reports/glerl-083/UpdatedFiles/ (downloaded October 12, 2026).

[6] T. S. Hunter, A. H. Clites, K. B. Campbell and A. D. Gronewold. Development and application of a North American Great Lakes hydrometeorological database, Part I: Precipitation, evaporation, runoff, and air temperature. *Journal of Great Lakes Research*, 41(1):65–77, 2015. https://doi.org/10.1016/j.jglr.2014.12.006

[7] GLISA, the NOAA Great Lakes CAP/RISA Team. Huron - Monthly - GLISA - General (dataset glisa_general_monthly_huron). Great Lakes Observing System, Seagull ERDDAP server. https://seagull-erddap.glos.org/erddap/info/glisa_general_monthly_huron/index.html. Data file: https://seagull-erddap.glos.org/erddap/tabledap/glisa_general_monthly_huron.csv (downloaded October 12, 2026).

[8] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*, 3rd edition. OTexts, Melbourne, Australia, 2021. https://otexts.com/fpp3/

[9] P. Virtanen, R. Gommers, T. E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. https://doi.org/10.1038/s41592-019-0686-2

[10] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. https://doi.org/10.1214/aos/1013203451

[11] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone. *Classification and Regression Trees*. Routledge, 2017. https://doi.org/10.1201/9781315139470

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[13] J. H. Friedman. Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4):367–378, 2002. https://doi.org/10.1016/S0167-9473(01)00065-2

[14] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. https://doi.org/10.1109/5.726791

[15] A. Paszke, S. Gross, F. Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[16] L. Prechelt. Early stopping – but when? In G. B. Orr and K.-R. Müller, editors, *Neural Networks: Tricks of the Trade*, Lecture Notes in Computer Science, pages 55–69. Springer, 1998. https://doi.org/10.1007/3-540-49430-8_3

[17] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv:1711.05101, 2017. Published as a conference paper at the International Conference on Learning Representations (ICLR), 2019. https://arxiv.org/abs/1711.05101

[18] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994. https://doi.org/10.1201/9780429246593

[19] H. R. Künsch. The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3):1217–1241, 1989. https://doi.org/10.1214/aos/1176347265

[20] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. https://doi.org/10.1023/A:1010933404324

[21] W. McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. https://doi.org/10.25080/Majora-92bf1922-00a

[22] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. https://doi.org/10.1038/s41586-020-2649-2

[23] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. https://doi.org/10.1109/MCSE.2007.55

A Code Listing

The following code implements the core of the pipeline described in this paper. The full code is split across three notebooks: `function_declaration.ipynb` defines every function and class, `data_processing.ipynb` builds the monthly table from the raw files, and `main.ipynb` runs the backtest and makes the figures. All of the code is written in Python, using pandas [21] and NumPy [22] for the data, SciPy [9] for fitting the lake model, scikit-learn [12] for gradient boosting, PyTorch [15] for the CNN and Matplotlib [23] for the figures.

```
import copy
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from scipy.optimize import least_squares
from sklearn.ensemble import GradientBoostingRegressor
from torch import nn


ROOT = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "data" / "raw").is_dir())
RAW = ROOT / "data" / "raw"
PROCESSED = ROOT / "data" / "processed"
FIGURES = ROOT / "figures"


AREA_MIC_KM2 = 57_800.0
AREA_HUR_KM2 = 59_600.0
AREA_COMBINED_M2 = (AREA_MIC_KM2 + AREA_HUR_KM2) * 1e6

START = "1950-01"


def read_depth_table(name):
    wide = pd.read_csv(RAW / name, comment="#", skipinitialspace=True)
    wide.columns = range(1, 13)
    long = wide.stack()
    long.index = pd.PeriodIndex(
        [f"{year}-{month:02d}" for year, month in long.index], freq="M"
    )
    return long


def read_runoff(name):
    df = pd.read_csv(RAW / name, sep=r"\s*,\s*", engine="python")
    df.index = pd.PeriodIndex(df["YYYY-MM"], freq="M")
    return df[["flow_cms", "fraction_gauged"]]


def read_levels():
    df = pd.read_csv(RAW / "levels_michigan_huron.csv", skiprows=[1])
    idx = pd.PeriodIndex(
        [f"{y}-{m:02d}" for y, m in zip(df["Year"], df["Month"])], freq="M"
    )
    return pd.Series(df["Water_Level_m"].to_numpy(), index=idx)


def runoff_to_depth_mm(flow_cms):
    seconds_in_month = flow_cms.index.days_in_month * 86_400
    return flow_cms * seconds_in_month / AREA_COMBINED_M2 * 1000.0


def build_monthly():
    w_mic = AREA_MIC_KM2 / (AREA_MIC_KM2 + AREA_HUR_KM2)
    w_hur = 1.0 - w_mic

    # Precip and evap are depths over each lake -> area-weighted average.
```

```
    precip = w_mic * read_depth_table("prc_mic_lake.csv") + w_hur * read_depth_table(
        "prc_hur_lake.csv"
    )
    evap = w_mic * read_depth_table("evaporation_mic.csv") + w_hur * read_depth_table(
        "evaporation_hur.csv"
    )

    # Runoff is a volume flux -> add the two basins, then convert to depth.
    ro_mic = read_runoff("runoff_mic_arm.csv")
    ro_hur = read_runoff("runoff_hur_arm.csv")
    runoff = runoff_to_depth_mm(ro_mic["flow_cms"] + ro_hur["flow_cms"])

    df = pd.DataFrame(
        {
            "level_m": read_levels(),
            "precip_mm": precip,
            "evap_mm": evap,
            "runoff_mm": runoff,
            "frac_gauged_mic": ro_mic["fraction_gauged"],
            "frac_gauged_hur": ro_hur["fraction_gauged"],
        }
    )
    df["nbs_mm"] = df["precip_mm"] - df["evap_mm"] + df["runoff_mm"]
    # Target: change in the monthly mean level from the previous month, in mm.
    df["dlevel_mm"] = df["level_m"].diff() * 1000.0

    df = df.loc[START:]
    # Keep only the span where every series is observed.
    complete = df[["level_m", "precip_mm", "evap_mm", "runoff_mm"]].notna().all(axis=1)
    df = df.loc[: complete[complete].index.max()]
    assert df.notna().all().all(), "unexpected gap inside the analysis period"
    df.index.name = "month"
    return df
```

Listing 1: Imports, data loading and the merged monthly table

```
COMPONENTS = ["precip_mm", "evap_mm", "runoff_mm"]
LEVEL_REF_M = 176.5  # only centres the level term; has no effect on forecasts


def month_of(index):
    return index.month.to_numpy()


class Persistence:
    def fit(self, train):
        return self

    def predict(self, df, origins, horizons):
        return np.zeros((len(origins), len(horizons)))


class SeasonalClimatology:
    def fit(self, train):
        self.mean_change_ = train["dlevel_mm"].groupby(month_of(train.index)).mean()
        return self

    def predict(self, df, origins, horizons):
        steps = self.mean_change_.to_numpy()[(origins.month.to_numpy()[:, None] + np.arange(max(horizons))) %
     12]
        return np.cumsum(steps, axis=1)[:, np.array(horizons) - 1]
```

Listing 2: Baseline models: persistence and seasonal climatology

```
class WaterBalance:
    def __init__(self, perfect_forcing=False):
        self.perfect_forcing = perfect_forcing
```

```

    def _design(self, month, x_now, x_prev, level_prev):
        dummies = np.zeros((len(month), 12))
        dummies[np.arange(len(month)), month - 1] = 1.0
        return np.column_stack([dummies, x_now, x_prev, level_prev - LEVEL_REF_M])

    def fit(self, train):
        x = train[COMPONENTS].to_numpy()
        design = self._design(month_of(train.index)[1:], x[1:], x[:-1], train["level_m"].to_numpy()[:-1])
        self.coef_, *_ = np.linalg.lstsq(design, train["dlevel_mm"].to_numpy()[1:], rcond=None)
        return self._fit_forcing(train)

    def _fit_forcing(self, train):
        self.clim_ = train[COMPONENTS].groupby(month_of(train.index)).mean()
        anom = train[COMPONENTS].to_numpy() - self.clim_.loc[month_of(train.index)].to_numpy()
        self.phi_ = np.array([np.corrcoef(a[1:], a[:-1])[0, 1] for a in anom.T])
        return self

    def _forcing(self, df, origins, k, observed):
        if observed:
            return df.loc[origins + k, COMPONENTS].to_numpy()
        anom = df.loc[origins, COMPONENTS].to_numpy() - self.clim_.loc[origins.month].to_numpy()
        return self.clim_.loc[(origins + k).month].to_numpy() + self.phi_**k * anom

    def predict(self, df, origins, horizons):
        level, x_prev, cum, path = df.loc[origins, "level_m"].to_numpy(), df.loc[origins, COMPONENTS].to_numpy
     (), 0, []
        for k in range(1, max(horizons) + 1):
            x_now = self._forcing(df, origins, k, self.perfect_forcing)
            cum = cum + self._design((origins + k).month.to_numpy(), x_now, x_prev, level + cum / 1000) @ self
     .coef_
            path.append(cum)
            x_prev = x_now
        return np.column_stack(path)[:, np.array(horizons) - 1]

    def coefficients(self):
        names = [f"month_{m}" for m in range(1, 13)]
        names += [f"{c}_lag0" for c in COMPONENTS] + [f"{c}_lag1" for c in COMPONENTS]
        names += ["level_m"]
        return pd.Series(self.coef_, index=names)
```

Listing 3: Water-balance model

```
def lake_supply(p, x, month):
    return p[0] * x[..., 0] - p[1] * x[..., 1] + p[2] * x[..., 2] + p[3:15][month - 1]


def lake_route(p, level, supply, substeps=10):
    def rate(level, s):
        return s - p[15] * (((level - LEVEL_REF_M + p[16]) / p[16]) ** 1.5 - 1)

    level = level + rate(level, supply[:, 0]) / 2000  # half a month: monthly mean -> end of month
    means = []
    for k in range(1, supply.shape[1]):
        total = 0
        for _ in range(substeps):
            level = level + rate(level, supply[:, k]) / (1000 * substeps)
            total = total + level / substeps
        means.append(total)
    return means


class LakeModel(WaterBalance):
    # p = [mP, mE, mR, n[1..12], c, d]; d >= 1.5 m keeps the sill z0 below the 2013 record low
    bounds = (np.r_[0, 0, 0, [-np.inf] * 12, 0, 1.5], np.r_[3, 3, 3, [np.inf] * 12, np.inf, 50])

    def fit(self, train):
        self._fit_forcing(train)
```

```
        origins = train.index[: -max(HORIZONS)]
        y = np.column_stack([_future_change(train, h).loc[origins] for h in HORIZONS])
        args = self._inputs(train, origins, max(HORIZONS), observed=True)
        # Start: no scaling, n = average budget gap per month, slope 1.5*c/d = 24 mm/month per m (as the water balance)
        channel = (train["dlevel_mm"] - train["nbs_mm"]).groupby(month_of(train.index)).mean()
        self.p_ = least_squares(lambda p: (self._changes(p, *args) - y).ravel(),
                                np.r_[1, 1, 1, channel, 80, 5], bounds=self.bounds).x
        return self

    def _inputs(self, df, origins, n_months, observed):
        x = np.stack([df.loc[origins, COMPONENTS].to_numpy()]
                     + [self._forcing(df, origins, k, observed) for k in range(1, n_months + 1)], axis=1)
        month = np.column_stack([(origins + k).month for k in range(n_months + 1)])
        return df.loc[origins, "level_m"].to_numpy(), x, month

    def _changes(self, p, level, x, month):
        return (np.column_stack(lake_route(p, level, lake_supply(p, x, month))) - level[:, None]) * 1000

    def predict(self, df, origins, horizons):
        return self._changes(self.p_, *self._inputs(df, origins, max(horizons), self.perfect_forcing))[:, np.array(horizons) - 1]
```

Listing 4: Lake model

```
def make_features(df):
    f = pd.DataFrame(index=df.index)
    f["level"] = df["level_m"] - LEVEL_REF_M
    for lag in range(3):
        f[f"dlevel_lag{lag}"] = df["dlevel_mm"].shift(lag)
        for c in COMPONENTS:
            f[f"{c[:-3]}_lag{lag}"] = df[c].shift(lag)
    for window in (3, 6, 12):
        f[f"nbs_sum{window}"] = df["nbs_mm"].rolling(window).sum()
    angle = 2 * np.pi * month_of(df.index) / 12
    f["month_sin"] = np.sin(angle)
    f["month_cos"] = np.cos(angle)
    return f


def _future_change(df, h):
    return (df["level_m"].shift(-h) - df["level_m"]) * 1000.0


class GradientBoosting:
    params = dict(n_estimators=300, learning_rate=0.03, max_depth=3,
                  subsample=0.8, min_samples_leaf=5, random_state=0)

    def __init__(self, base=None):
        self.base = base

    def fit(self, train):
        self.models_ = {}
        feats = make_features(train)
        ok = feats.notna().all(axis=1)
        if self.base is not None:
            self.base.fit(train)
        for h in range(1, 7):
            y = _future_change(train, h)
            rows = ok & y.notna()
            origins = train.index[rows]
            target = y[rows].to_numpy()
            if self.base is not None:
                target = target - self.base.predict(train, origins, [h])[:, 0]
            self.models_[h] = GradientBoostingRegressor(**self.params).fit(
                feats.loc[origins], target
            )
        return self
```

```
    def predict(self, df, origins, horizons):
        # Features are backward-looking, so computing them on the full table
        # does not leak future information into a given origin's row.
        feats = make_features(df).loc[origins]
        out = np.column_stack([self.models_[h].predict(feats) for h in horizons])
        if self.base is not None:
            out = out + self.base.predict(df, origins, horizons)
        return out
```

Listing 5: Features, gradient boosting and the hybrid

```
class PhysicsCNN:
    window, epochs, lr = 24, 300, 1e-3

    def fit(self, train):
        torch.manual_seed(0)
        self.lake_ = LakeModel().fit(train)
        channels = self._channels(train)
        self.mu_, self.sd_ = channels.mean(), channels.std()
        anom = train[COMPONENTS] - self.lake_.clim_.loc[month_of(train.index)].to_numpy()
        self.xsd_ = torch.tensor(anom.std().to_numpy())
        origins = train.index[self.window - 1 : -max(HORIZONS)]
        level, x, month = map(torch.tensor, self.lake_._inputs(train, origins, max(HORIZONS), observed=True))
        x_ar1 = torch.tensor(self.lake_._inputs(train, origins, max(HORIZONS), observed=False)[1][:, 1:])
        y = torch.tensor(np.column_stack([_future_change(train, h).loc[origins] for h in HORIZONS]))
        windows = self._windows(train, origins)
        self.net_ = nn.Sequential(nn.Conv1d(channels.shape[1], 8, 3), nn.ReLU(), nn.Conv1d(8, 8, 3), nn.ReLU()
    ,
                                  nn.Flatten(), nn.Linear(8 * (self.window - 4), 3 * max(HORIZONS))).double()
        for w in self.net_[-1].parameters():  # start as the lake model's own forecast
            nn.init.zeros_(w)
        opt = torch.optim.AdamW(self.net_.parameters(), lr=self.lr, weight_decay=1e-2)

        def loss(rows):  # level error + weather error, each scaled by its own spread
            change, x_future = self._forward(windows[rows], level[rows], x[rows, 0], x_ar1[rows], month[rows])
            return (((change - y[rows]) / y.std(0)) ** 2).mean() + (((x_future - x[rows, 1:]) / self.xsd_) **
    2).mean()

        fit_rows, val_rows = slice(None, int(0.8 * len(origins))), slice(int(0.8 * len(origins)), None)
        best, state = loss(val_rows).item(), copy.deepcopy(self.net_.state_dict())
        for _ in range(self.epochs):
            opt.zero_grad()
            loss(fit_rows).backward()
            opt.step()
            with torch.no_grad():
                val = loss(val_rows).item()
            if val < best:
                best, state = val, copy.deepcopy(self.net_.state_dict())
        self.net_.load_state_dict(state)
        return self

    def _channels(self, df):
        return pd.concat([df[COMPONENTS + ["dlevel_mm", "level_m"]], make_features(df)[["month_sin", "
    month_cos"]]], axis=1)

    def _windows(self, df, origins):
        z = ((self._channels(df) - self.mu_) / self.sd_).to_numpy()
        return torch.tensor(z[df.index.get_indexer(origins)[:, None] + np.arange(1 - self.window, 1)]).
    transpose(1, 2)

    def _forward(self, windows, level, x_origin, x_ar1, month):
        p = torch.tensor(self.lake_.p_)
        x_future = x_ar1 + self.net_(windows).view(len(level), -1, 3) * self.xsd_
        supply = lake_supply(p, torch.cat([x_origin[:, None], x_future], 1), month)
        return (torch.stack(lake_route(p, level, supply), 1) - level[:, None]) * 1000, x_future

    def predict(self, df, origins, horizons):
```

```
        level, x, month = map(torch.tensor, self.lake_._inputs(df, origins, max(HORIZONS), observed=False))
        with torch.no_grad():
            change, _ = self._forward(self._windows(df, origins), level, x[:, 0], x[:, 1:], month)
        return change.numpy()[:, np.array(horizons) - 1]


def all_models():
    return {
        "Persistence": Persistence(),
        "Seasonal climatology": SeasonalClimatology(),
        "Water balance": WaterBalance(),
        "Lake model": LakeModel(),
        "Gradient boosting": GradientBoosting(),
        "Hybrid (WB + GB residual)": GradientBoosting(base=WaterBalance()),
        "Physics-guided CNN": PhysicsCNN(),
        "WB, perfect forcing (upper bound)": WaterBalance(perfect_forcing=True),
        "Lake model, perfect forcing (upper bound)": LakeModel(perfect_forcing=True),
    }
```

Listing 6: Physics-guided CNN and the list of models

```
HORIZONS = [1, 2, 3, 4, 5, 6]
FIRST_TEST_YEAR = 1980  # leaves 30 years (1950-79) for the first training set


def backtest(df, first_test_year=FIRST_TEST_YEAR, verbose=True):
    rows = []
    for year in range(first_test_year, df.index.max().year + 1):
        train = df.loc[: f"{year - 1}-12"]
        targets = df.loc[f"{year}-01":f"{year}-12"].index
        models = {name: m.fit(train) for name, m in all_models().items()}
        for h in HORIZONS:
            origins = targets - h
            start = df.loc[origins, "level_m"].to_numpy()
            observed = df.loc[targets, "level_m"].to_numpy()
            for name, model in models.items():
                change_mm = model.predict(df, origins, [h])[:, 0]
                rows.append(pd.DataFrame({
                    "model": name, "target": targets, "origin": origins, "horizon": h,
                    "observed_m": observed, "predicted_m": start + change_mm / 1000.0,
                }))
        if verbose:
            print(f"  tested {year}", end="\r")
    preds = pd.concat(rows, ignore_index=True)
    preds["error_cm"] = (preds["predicted_m"] - preds["observed_m"]) * 100.0
    return preds


def rmse_table(preds):
    rmse = preds.groupby(["model", "horizon"], sort=False)["error_cm"].apply(
        lambda e: np.sqrt(np.mean(e**2))
    )
    return rmse.unstack("horizon")


def bootstrap_rmse_diff(preds, model_a, model_b, n_boot=2000, seed=0):
    rng = np.random.default_rng(seed)
    out = {}
    for h in HORIZONS:
        sq = {}
        for name in (model_a, model_b):
            p = preds[(preds["model"] == name) & (preds["horizon"] == h)]
            sq[name] = (p["error_cm"] ** 2).groupby(p["target"].dt.year).agg(["sum", "count"])
        years = sq[model_a].index.to_numpy()
        a, b = sq[model_a].to_numpy(), sq[model_b].to_numpy()

        def diff(idx):
            return (np.sqrt(a[idx, 0].sum() / a[idx, 1].sum())
```

```
                    - np.sqrt(b[idx, 0].sum() / b[idx, 1].sum()))

        boots = [diff(rng.integers(0, len(years), len(years))) for _ in range(n_boot)]
        lo, hi = np.percentile(boots, [2.5, 97.5])
        out[h] = {"diff_cm": diff(np.arange(len(years))), "ci_low": lo, "ci_high": hi}
    return pd.DataFrame(out).T.rename_axis("horizon")
```

Listing 7: Rolling-origin backtest, RMSE and the bootstrap

```
# Feature groups used for the permutation-importance figure.
FEATURE_GROUPS = {
    "Season (month)": ["month_sin", "month_cos"],
    "Current level": ["level"],
    "Recent level change": [f"dlevel_lag{k}" for k in range(3)],
    "Precipitation": [f"precip_lag{k}" for k in range(3)],
    "Evaporation": [f"evap_lag{k}" for k in range(3)],
    "Runoff": [f"runoff_lag{k}" for k in range(3)],
    "Accumulated NBS (3-12 mo)": ["nbs_sum3", "nbs_sum6", "nbs_sum12"],
}


def permutation_importance(df, split_year=2010, n_repeats=20, seed=0):
    rng = np.random.default_rng(seed)
    model = GradientBoosting().fit(df.loc[: f"{split_year - 1}-12"])
    feats = make_features(df)
    rows = {}
    for h in (1, 3, 6):
        targets = df.loc[f"{split_year}-01":].index
        origins = targets - h
        x = feats.loc[origins]
        y = (df.loc[targets, "level_m"].to_numpy() - df.loc[origins, "level_m"].to_numpy()) * 1000

        def rmse_cm(frame):
            return np.sqrt(np.mean((model.models_[h].predict(frame) - y) ** 2)) / 10

        base = rmse_cm(x)
        for group, cols in FEATURE_GROUPS.items():
            increases = []
            for _ in range(n_repeats):
                shuffled = x.copy()
                shuffled[cols] = x[cols].to_numpy()[rng.permutation(len(x))]
                increases.append(rmse_cm(shuffled) - base)
            rows[(h, group)] = np.mean(increases)
    return pd.Series(rows).unstack(0)
```

Listing 8: Permutation importance